

DEEP REINFORCEMENT LEARNING · INCREMENTAL EXAMPLE

Race Car: MDP → DP → MC → TD

One tiny 3-state problem, solved four ways — so you can see exactly how known-model planning, episode sampling, and one-step bootstrapping differ.

Worked example deck · Read alongside the lecture slides · Every number on the slides reproduced in full

How to use this companion. This deck takes *one* problem — a race car managing its engine temperature — and solves it with every major method, so the differences are concrete, not abstract. Colour boxes guide you: **blue** intuition, **amber** everyday picture, **green** worked example, **red** watch out, **purple** key takeaway. Every slide computation is reproduced step by step.

1 The race-car problem

A race car completes laps while managing **engine temperature**. At each step it chooses **Go Slow** or **Go Fast**, trading lap time against overheating risk. The engine has three states: **Cool**, **Warm**, and **Overheated** (a terminal “shutdown”). The next temperature depends only on the current state — the **Markov property**.

The intuition

Going fast earns more reward but heats the engine; going slow is safer but slower. The agent must learn a **policy** — which speed to pick in each temperature — that maximises long-run reward without overheating.

1.1 The MDP

- **States** $S = \{\text{Cool}, \text{Warm}, \text{Overheated}\}$.
- **Actions** $A = \{\text{Slow}, \text{Fast}\}$.
- **Dynamics & rewards** (the version that reproduces the slide numbers, $\gamma = 0.9$ for DP): *Slow* from Cool → Cool, reward +1; *Fast* from Cool → Cool or Warm (each 0.5), reward +2; *Slow* from Warm → Cool or Warm (each 0.5), reward +1; *Fast* from Warm → Overheated, reward −10.

Key takeaway

The race car is a 3-state, 2-action MDP: Fast earns more but risks overheating. We will solve it four ways and watch each method recover the same optimal policy.

2 Dynamic Programming (known model)

When the model $p(s', r | s, a)$ is *known*, we can plan exactly with DP, as in the previous session. Two algorithms apply.

2.1 Value iteration

Repeatedly apply the Bellman *optimality* backup $V_{k+1}(s) \leftarrow \max_a \sum_{s', r} p(s', r | s, a)[r + \gamma V_k(s')]$ until convergence, then read off the greedy policy.

Worked example: value iteration, sweep by sweep (slide numbers)

Start $V_0 = (\text{Cool } 0, \text{ Warm } 0, \text{ Overheated } 0)$, $\gamma = 0.9$.

Sweep 1. $V_1(\text{Cool}) = \max(\underbrace{1 + 0.9(0)}_{\text{Slow}=1}, \underbrace{2 + 0.9(0)}_{\text{Fast}=2}) = 2$; $V_1(\text{Warm}) = \max(1 + 0.9(0), -10) = 1$.

So $V_1 = (2, 1, 0)$.

Sweep 2. $V_2(\text{Cool}) = \max(1 + 0.9(2) = 2.8, 2 + 0.9(0.5 \cdot 2 + 0.5 \cdot 1) = 3.35) = 3.35$; $V_2(\text{Warm}) = \max(1 + 0.9(1.5) = 2.35, -10) = 2.35$. So $V_2 = (3.35, 2.35, 0)$.

Sweep 3. Continuing, the synchronous sweeps converge near $(15.41, 14.41, 0)$ when the change $\Delta \approx 0.009 < \theta = 0.01$ (about **49** sweeps to reach the threshold, matching the deck's ~ 46); the exact fixed point is **(15.5, 14.5, 0)**.

Sweep 4. Extract policy: Cool $\rightarrow$ **Fast**, Warm $\rightarrow$ **Slow**.

Worked example: asynchronous is faster (slide numbers)

Asynchronous value iteration updates in place, reusing the fresh $V(\text{Cool}) = 2$ while updating $V(\text{Warm})$ in the *same* sweep.

Step 1. Sweep 2 (async): $V_2 = (3.75, 3.54, 0)$ — higher than the synchronous $(3.35, 2.35, 0)$, because Warm used the updated Cool value.

Step 2. Converges near $(15.44, 14.44, 0)$ in about **40** sweeps, versus 49 synchronous. Fresher values, faster convergence.

2.2 Policy iteration

Alternate full **policy evaluation** (Bellman *expectation* backups for a fixed policy) and **policy improvement** (greedy update).

 Worked example: policy iteration on the race car (slide numbers)

Start $\pi_0 = (\text{Cool: Slow, Warm: Slow})$.

Step 1. Evaluate π_0 . Backups under the fixed policy converge near $V_{225} \approx (10, 10, 0)$.

Step 2. Improve. Taking the max-value action gives $\pi_1 = (\text{Cool: Fast, Warm: Slow})$.

Step 3. Repeat one cycle. Converges with $V(\text{Cool}) \approx 15.5$, $V(\text{Warm}) \approx 14.5$; the policy no longer changes. Optimal in just **3** policy steps (vs ~ 49 value-iteration sweeps).

 Watch out

DP found the answer without ever “driving” the car — it used the known model. The moment the model is unknown, DP cannot be applied; we must *sample* experience instead. That is the rest of this deck.

✓ Key takeaway

With a known model, DP plans the optimal policy exactly: value iteration (max sweeps, ~ 23) or policy iteration (evaluate + improve, ~ 3 steps). Both recover Cool→Fast, Warm→Slow.

3 Monte Carlo (unknown model, full episodes)

Now the model is *unknown*: the agent does not know the transition probabilities. **Monte Carlo (MC)** methods learn by *sampling complete episodes* and averaging the actual **returns**. They need no model — only experience — but must wait until an episode *ends* to learn from it.

🧠 The intuition

MC is “learn from the final score.” Drive a whole lap (an episode), record the total discounted reward from each state-action you visited, and average those returns over many laps. No bootstrapping, no model — just real outcomes.

★ Everyday picture

Like rating restaurants only after a *complete* meal. You do not guess mid-meal; you wait for the bill, total your enjoyment, and update your average for that place. Slow to learn, but grounded entirely in what actually happened.

3.1 On-policy first-visit MC

For each state-action pair, average the returns *following the first time it is visited* in each episode. The return is computed backwards: $G_t = R_{t+1} + \gamma G_{t+1}$.

 Worked example: first-visit returns for Episode 1 (slide numbers, $\gamma = 0.6$)

Episode 1: $C, F, 2, W, S, 1, C, F, 2, W, F, -10, O$. The rewards in time order are $R_1 = 2, R_2 = 1, R_3 = 2, R_4 = -10$. Compute returns backwards.

Step 1. $G_3 = -10$ (the last reward; next is terminal).

Step 2. $G_2 = R_3 + \gamma G_3 = 2 + 0.6(-10) = 2 - 6 = -4$.

Step 3. $G_1 = R_2 + \gamma G_2 = 1 + 0.6(2) + 0.36(-10) = 1 + 1.2 - 3.6 = -1.4$. (Equivalently $1 + 0.6(-4) = -1.4$.)

Step 4. $G_0 = R_1 + \gamma G_1 = 2 + 0.6(1) + 0.36(2) + 0.216(-10) = 2 + 0.6 + 0.72 - 2.16 = 1.16$.

Step 5. Assign to first visits. $Q(C, F) \leftarrow G_0 = 1.16$ (first visit of (C, F) at $t=0$); $Q(W, S) \leftarrow G_1 = -1.4$; $Q(W, F) \leftarrow G_3 = -10$. The second (C, F) at $t=2$ with $G_2 = -4$ is *ignored* under first-visit.

After improvement, the policy shifts toward the safer Slow action — exactly the slide's π_1 .

 Watch out

First-visit MC uses only the *first* occurrence of a state-action per episode; **every-visit** MC averages *all* occurrences. Here, the $t=2$ visit of (C, F) with return -4 is dropped by first-visit but kept by every-visit — they can give different estimates.

3.2 Off-policy MC and importance sampling

Off-policy MC learns about a **target** policy π while following a different **behaviour** policy b (e.g. a more exploratory one, or logged data). To correct for the mismatch, it weights returns by the **importance-sampling ratio** $W = \prod \frac{\pi(A_t|S_t)}{b(A_t|S_t)}$.

There are two flavours. **Ordinary importance sampling (OIS)** is unbiased but *high variance*. **Weighted importance sampling (WIS)** normalises by the sum of weights — biased but *much lower variance*, which is usually preferred.

 Worked example: one importance-sampling weight (slide numbers)

With a deterministic target ($\pi(A_t | S_t) = 1$ when the action matches) and a stochastic behaviour b , at a step where the action taken matches the target and $b(\text{Slow} | \text{Warm}) = 0.4$:

Step 1. $W = \frac{\pi(\text{Slow} | \text{Warm})}{b(\text{Slow} | \text{Warm})} = \frac{1}{0.4} = \mathbf{2.5}$.

Step 2. This W scales the return when updating Q . If at some step the action *does not* match the target ($\pi = 0$), the trajectory's weight becomes 0 and the algorithm stops updating from that point back — the experience is “off-target” and discarded.

 Watch out

Off-policy MC only learns from the *tail* of an episode where behaviour and target agree — once they diverge (a target-probability-zero action), the weight is 0 and earlier steps get no update. This is why off-policy MC can be slow and high-variance, and why WIS (normalised) is the practical choice.

✓ Key takeaway

Monte Carlo learns model-free by averaging *complete-episode returns*: first-visit (or every-visit) on-policy, and importance-sampled off-policy (OIS unbiased/high-var, WIS biased/low-var). The cost: it must wait for episodes to finish.

4 Temporal-Difference learning (one-step bootstrap)

Temporal-Difference (TD) learning is the best of both worlds: model-free like MC, but it learns from *one step*, like DP — without waiting for the episode to end. It **bootstraps**: it updates an estimate toward a target built from the *next* estimate.

🔍 The intuition

MC waits for the final score; DP needs the model; TD updates after *every* step using its own current guess of the next state's value. It is the Tic-Tac-Toe rule from Session 1, now with rewards: $V(S_t) \leftarrow V(S_t) + \alpha[R + \gamma V(S_{t+1}) - V(S_t)]$.

4.1 TD(0) prediction

The **TD(0)** update for state values:

$$V(S_t) \leftarrow V(S_t) + \alpha \underbrace{[R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]}_{\text{TD target}}. \quad (1)$$

📎 Worked example: TD(0) updates (slide numbers, $\gamma = 0.9$, $\alpha = 0.5$)

Episode 1: $C, F, 2, W, \dots$, starting from $V = (0, 0, 0)$.

Step 1. First transition $C \xrightarrow{2} W$. Target = $R + \gamma V(W) = 2 + 0.9(0) = 2$. TD error = $2 - V(C) = 2 - 0 = 2$. Update $V(C) = 0 + 0.5(2) = 1$.

Step 2. Next transition $W \xrightarrow{1} C$. $V(W) = 0 + 0.5[1 + 0.9(1) - 0] = 0.5(1.9) = 0.95$.

Step 3. So what. After just one short episode, $V(C) = 1$ and $V(W) = 0.95$ — learning happened *during* the episode, no waiting for the end.

4.2 SARSA: on-policy TD control

SARSA learns action-values on-policy. Its name is its update tuple $(S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1})$:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha[R + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)]. \quad (2)$$

It uses the action A_{t+1} that the policy *actually* takes next.

 Worked example: SARSA updates (slide numbers, $\gamma = 0.9$, $\alpha = 0.8$)

Episode 1 step $(C, F, 2, W, S)$, from $Q = 0$.

Step 1. $Q(C, F) = 0 + 0.8[2 + 0.9Q(W, S) - Q(C, F)] = 0 + 0.8[2 + 0.9(0) - 0] = \mathbf{1.6}$.

Step 2. Next step $(W, S, 1, C, F)$: $Q(W, S) = 0 + 0.8[1 + 0.9Q(C, F) - 0] = 0.8[1 + 0.9(1.6)] = 0.8(2.44) = \mathbf{1.95}$.

Step 3. Later, $(W, F, -10, O)$: $Q(W, F) = 0 + 0.8[-10 + 0.9(0) - 0] = -8$ — SARSA *feels* the danger of going Fast while Warm.

4.3 Expected SARSA

Expected SARSA replaces the single next action with its *expectation* over the policy, reducing variance:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R + \gamma \sum_a \pi(a | S_{t+1}) Q(S_{t+1}, a) - Q(S_t, A_t) \right]. \quad (3)$$

 Worked example: Expected SARSA (slide numbers, equiprobable next actions)

Step $(C, F, 2, W)$ with both Warm-actions equally likely.

Step 1. Expected next value $= 0.5Q(W, S) + 0.5Q(W, F) = 0.5(0) + 0.5(0) = 0$.

Step 2. $Q(C, F) = 0 + 0.8[2 + 0.9(0) - 0] = \mathbf{1.6}$ (same as SARSA on step one, since the next Q s are still 0).

Step 3. The benefit appears later: averaging over next actions instead of sampling one cuts the variance that made plain SARSA's $Q(W, S)$ jump around.

 Watch out

SARSA is **on-policy**: it evaluates the policy it actually follows (exploration included), so it learns “safe” values that account for occasional random actions. Expected SARSA lowers variance by averaging the next step. Both can become off-policy variants — in fact, Expected SARSA with a greedy target *is* Q-learning.

4.4 Q-learning: off-policy TD control

Q-learning learns the *optimal* action-values directly, off-policy, by bootstrapping from the *best* next action (a max), regardless of what the behaviour policy actually did:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [R + \gamma \max_{a'} Q(S_{t+1}, a') - Q(S_t, A_t)]. \quad (4)$$

 Worked example: Q-learning update (slide numbers, $\gamma = 0.9$, $\alpha = 0.8$)

Step $(C, F, 2, W)$ from $Q = 0$.

Step 1. $Q(C, F) = 0 + 0.8[2 + 0.9 \max(Q(W, S), Q(W, F)) - 0] = 0.8[2 + 0.9(0)] = \mathbf{1.6}$.

Step 2. With a later large reward, e.g. $(C, F, 2, W, S, 20, C)$: $Q(W, S) = 0 + 0.8[20 + 0.9 \max(Q(C, S), Q(C, F)) - 0] \approx \mathbf{17.15}$ — the max pulls value from the best next action, not the one taken.

 Watch out

The max makes Q-learning learn the *optimal* policy even while *exploring* (behaving ε -greedily). But that same max causes **maximisation bias**: noise in the estimates makes the max systematically too high. The slide's $Q(W, S)$ swinging between successive episodes is this variance/bias at work.

4.5 Double Q-learning

Double Q-learning fixes maximisation bias by keeping *two* value tables and decoupling action *selection* from action *evaluation*: one table picks the best action, the other supplies its value.

$$Q_1(S_t, A_t) \leftarrow Q_1(S_t, A_t) + \alpha[R + \gamma Q_2(S_{t+1}, \arg \max_{a'} Q_1(S_{t+1}, a')) - Q_1(S_t, A_t)], \quad (5)$$

and symmetrically (swapping Q_1, Q_2) on alternate updates.

 Worked example: one double-Q update (slide numbers)

Step $(C, F, 2, W)$, with $Q_1(W, \cdot)$ available as behaviour values and Q_2 as the table being updated. Suppose $Q_1(W, \text{Fast}) = 5$ and the argmax over $Q_2(W, \cdot)$ selects Fast.

Step 1. Selection by Q_2 picks $\arg \max_a Q_2(W, a) = \text{Fast}$ (a tie broken to Fast).

Step 2. Evaluation by Q_1 : $Q_1(W, \text{Fast}) = 5$.

Step 3. $Q_2(C, F) = 0 + 0.8[2 + 0.9(5) - 0] = 0.8(6.5) = \mathbf{5.2}$. Because one table selects and the *other* evaluates, an over-optimistic estimate in one table does not inflate the update.

 Key takeaway

TD learns model-free *and* online: TD(0) for prediction; SARSA (on-policy) and Q-learning (off-policy, with a max) for control; Expected SARSA and Double Q-learning tame variance and maximisation bias. The update is always old + $\alpha[\text{TD target} - \text{old}]$.

5 DP vs MC vs TD — the unifying view

The same race car, three philosophies of *evaluation*. The **backup diagrams** say it all.

	DP	Monte Carlo	TD
Needs model?	Yes (full p)	No	No
Bootstraps?	Yes	No	Yes
Updates when?	Every sweep	End of episode	Every step
Backup width	All next states	One trajectory	One next state
Variance	— (exact)	High	Lower

Same problem, three backups: breadth (DP), depth (MC), one step (TD)

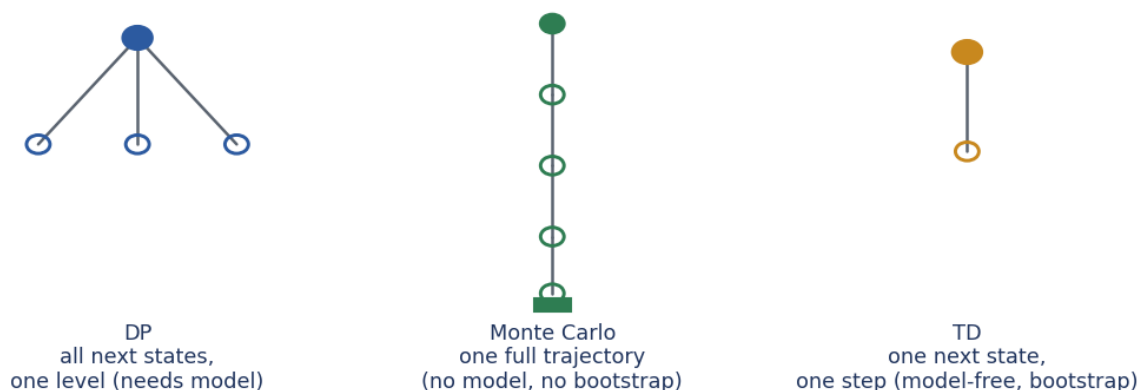


Figure 1: Backup diagrams. **DP** backs up from *all* next states (full breadth, one level deep — needs the model). **MC** backs up along *one full trajectory* to the end (one path, full depth — needs no model, no bootstrap). **TD** backs up from *one* next state, one step (narrow and shallow — model-free and bootstrapped).

The intuition

DP is the planner with the full map (uses the model). **MC** is the explorer who only trusts a completed journey (samples full returns). **TD** is the nimble learner who updates a guess from the next guess every step (samples and bootstraps). All three are instances of **Generalised Policy Iteration** — they differ only in *how* they evaluate.

Key takeaway

DP plans with the model; MC samples full-episode returns (no bootstrap); TD bootstraps one step (no model). Same GPI skeleton, three evaluation styles — and TD's blend of model-free sampling with one-step bootstrapping is the foundation of deep RL.

6 Recap: one problem, four lenses

We solved a single 3-state race-car MDP four ways. **DP** (value and policy iteration) used the known model to plan the optimum exactly — Cool→Fast, Warm→Slow. **Monte Carlo** dropped the model and learned from complete-episode returns (first-visit on-policy; importance-sampled off-policy). **TD** learned online from one-step bootstraps — TD(0) for prediction, SARSA and Q-learning for control, with Expected SARSA and Double Q-learning controlling variance and bias. The unifying picture: every method is Generalised Policy Iteration, differing only in how it evaluates — known dynamics (DP), sampled returns (MC), or bootstrapped one-step targets (TD). From here, deep RL replaces the value tables with neural networks, but the algorithms — and these exact updates — stay the same.